

AUTOSAR: AUTomotive Open System ARchitecture

Classic Platform & Adaptive Platform – Architecture Overview for ADAS Engineers

Knowledge Base Document · RAG-Ready · ADAS Domain

1. Introduction to AUTOSAR

AUTOSAR (AUTomotive Open System ARchitecture) is a worldwide development partnership of automotive OEMs, Tier 1 suppliers, semiconductor manufacturers, and tool vendors established in 2003. Its primary objective is to create and establish an open and standardised software architecture for automotive ECUs (Electronic Control Units), enabling hardware-independent software development and facilitating software reuse across vehicle models, OEMs, and suppliers.

AUTOSAR standardises the interfaces between application software and the underlying hardware, allowing software components to be developed independently of the hardware they will ultimately run on. This enables OEMs to source software components from multiple suppliers and integrate them on any AUTOSAR-compliant hardware platform, dramatically reducing integration effort and enabling software reuse.

1.1 The Two AUTOSAR Platforms

AUTOSAR currently maintains two distinct platforms addressing different use cases in modern vehicles:

- **Classic Platform (CP):** Designed for deeply embedded, real-time ECUs with deterministic timing requirements, resource constraints, and hard safety requirements. This is the mature platform used in powertrain, chassis, body, and traditional ADAS ECUs. Based on OSEK/VDX OS.
- **Adaptive Platform (AP):** Designed for high-performance computing platforms (HPC) running complex software such as automated driving stacks, sensor fusion, and AI/ML inference. Supports dynamic software loading, service-oriented communication, and runs on POSIX-based operating systems (e.g., QNX, Linux).

Characteristic	Classic Platform	Adaptive Platform
Primary Use	Safety ECUs, powertrain, body, traditional ADAS	ADAS HPC, autonomous driving, infotainment, gateway
OS	AUTOSAR OS (OSEK/VDX based)	POSIX OS (QNX, Linux, etc.)
Communication	COM stack, PDUs, CAN/LIN/FlexRay/Ethernet	Service-oriented, SOME/IP, DDS
SW Loading	Static at compile time	Dynamic at runtime
Memory	Typically < 1 MB RAM	GB-scale RAM, storage
Scheduling	Fixed cyclic, static	Dynamic, POSIX threads
Safety Support	ASIL A–D full support	ASIL B support, evolving
Language	C (primary)	C++14/17 (primary)
Maturity	Highly mature (R4.x–R25-11)	Evolving (R19-11–R25-11)

2. AUTOSAR Classic Platform Architecture

The Classic Platform defines a layered software architecture for deeply embedded automotive ECUs. The architecture strictly separates application software from the underlying hardware through standardised interfaces, enabling portability and reuse across different microcontrollers and ECU hardware.

2.1 Layered Architecture

The Classic Platform architecture consists of three main layers:

- **Application Layer:** Contains the AUTOSAR Software Components (SW-Cs) that implement the actual vehicle functions (e.g., ACC controller, AEB logic, body control). SW-Cs communicate through well-defined ports and interfaces, completely independent of hardware. SW-Cs are classified as Application SW-Cs, Sensor/Actuator SW-Cs, Complex Device Driver (CDD) SW-Cs, and Composition SW-Cs.
- **AUTOSAR Runtime Environment (RTE):** The middleware layer that mediates all communication between SW-Cs and between SW-Cs and the Basic Software. The RTE is generated from the system configuration (ARXML) by AUTOSAR tooling. It maps port-based SW-C communication to the appropriate BSW communication services at runtime. The RTE provides a hardware-independent communication API to all SW-Cs.
- **Basic Software (BSW):** Standardised low-level software modules providing services to the application. BSW is divided into: Microcontroller Abstraction Layer (MCAL), ECU Abstraction Layer, Services Layer, and Complex Drivers.

2.2 Basic Software (BSW) Modules

The BSW layer contains over 100 standardised module specifications. Key modules relevant to ADAS:

2.2.1 Microcontroller Abstraction Layer (MCAL)

- **ADC Driver:** Analogue-to-digital conversion for sensor inputs.
- **PWM Driver:** Pulse-width modulation for actuator control.
- **SPI Handler/Driver:** Serial peripheral interface for sensor communication.
- **FlexRay Driver, CAN Driver, LIN Driver, Ethernet Driver:** Bus-specific communication drivers.
- **GPT Driver:** General purpose timer for scheduling and timing functions.
- **WDG Driver:** Watchdog driver for hardware watchdog management (critical for ISO 26262).

2.2.2 Communication Stack

- **COM (Communication Manager):** Manages sending and receiving of signals and signal groups across the vehicle communication network. Supports signal filtering, invalidation, and timeout monitoring.
- **PDU Router (PduR):** Routes Protocol Data Units between communication modules and the transport layer. Enables flexible routing between CAN, LIN, FlexRay, and Ethernet.
- **CAN Interface (CanIf) / CAN SM / CAN NM:** CAN interface, state management, and network management.
- **SOME/IP Transformer:** Serialises/deserialises SOME/IP messages for service-oriented communication over Ethernet. Increasingly relevant as ADAS systems adopt Ethernet backbone communication.
- **E2E Library:** End-to-End communication protection — adds CRC checksums and sequence counters to safety-critical PDUs. Essential for ISO 26262 transmission error detection in ADAS systems.

2.2.3 Diagnostics

- **DEM (Diagnostic Event Manager):** Manages diagnostic events (faults), stores DTCs (Diagnostic Trouble Codes) in non-volatile memory, and provides fault status information to the application. ADAS systems report sensor faults, functional degradations, and safety violations through DEM.
- **DCM (Diagnostic Communication Manager):** Handles UDS (ISO 14229) diagnostic requests from external testers. Provides read/clear DTC, ECU reset, programming session management.

- **FIM (Function Inhibition Manager):** Controls which ADAS functions are active based on DEM fault status. When ACC radar reports a fault, FIM inhibits ACC functions and enables fallback behaviour.

2.2.4 Memory and OS Services

- **NVRAM Manager (NvM):** Manages persistent data storage in non-volatile memory (EEPROM/Flash). Stores calibration data, learned parameters, and DTC history.
- **AUTOSAR OS:** Real-time operating system based on OSEK/VDX. Provides tasks, alarms, resource management, and interrupt management. Supports static configuration at compile time. Tasks are assigned to one of four conformance classes (BCC1, BCC2, ECC1, ECC2).
- **Memory Protection:** AUTOSAR OS supports memory protection via MPU configuration, providing freedom-from-interference between application software components of different ASIL levels.

2.3 Software Component Model

AUTOSAR SW-Cs communicate exclusively through typed ports defined in the Software Component Description (SWC-Desc). Port types:

- **Sender-Receiver ports:** Asynchronous data exchange. One component sends data, others receive it. Most common pattern for signal-based ADAS communication (e.g., object list distribution).
- **Client-Server ports:** Synchronous or asynchronous function calls. Used for triggering actions (e.g., requesting a diagnostic operation, calling a complex calculation service).
- **Mode switch ports:** Used for mode management — components can query and respond to system mode changes (e.g., IGN_ON, ACC_ACTIVE, DEGRADED_MODE).

All port interfaces are defined in ARXML (AUTOSAR XML) and processed by AUTOSAR tooling to generate the RTE. This tooling-driven approach enforces interface discipline and enables automated consistency checking across the entire vehicle software configuration.

3. AUTOSAR Adaptive Platform Architecture

The Adaptive Platform (AP) was introduced in 2017 to address the requirements of high-performance computing applications in vehicles — particularly autonomous driving, sensor fusion, AI/ML inference, and over-the-air software updates. The AP is built on a POSIX-compliant OS and supports dynamic software loading and service-oriented communication.

3.1 AUTOSAR Runtime for Adaptive Applications (ARA)

The Adaptive Platform defines the ARA as the standardised interface between Adaptive Applications (AA) and the platform services. ARA consists of two types of interfaces:

- **ara::com:** Service-oriented communication API. Adaptive Applications discover and communicate with services dynamically at runtime. Supports both SOME/IP and local inter-process communication. Eliminates the need for static signal-based configuration typical of the Classic Platform.
- **ara::exec, ara::log, ara::diag, ara::per, ara::crypto, ara::phm, ara::ucm:** APIs for execution management, logging/tracing, diagnostics, persistent storage, cryptography, platform health management, and update/configuration management respectively.

3.2 Functional Clusters

The Adaptive Platform is organised into Functional Clusters — groups of related services:

Functional Cluster	Abbreviation	Purpose
Execution Management	EM	Controls lifecycle of Adaptive Applications. Starts, stops, restarts applications.
Communication Management	CM	Service-oriented communication via ara::com. SOME/IP and local binding support.
Diagnostics	DM	UDS-based diagnostics, DTC management, DoIP (Diagnostics over IP).
Persistency	PER	Persistent key-value storage and file storage for Adaptive Applications.
Cryptography	CRYPT	Cryptographic operations, key management, TLS support.
Platform Health Management	PHM	Monitors application health. Supervised entities must report alive/deadline/logical supervisory
Time Synchronization	TS	Provides global time to applications via gPTP (IEEE 802.1AS).
Update & Config Management	UCM	Over-the-air (OTA) software update management.
Network Management	NM	Network management for Ethernet-based vehicle networks.
Intrusion Detection System Mgr	IDS	Manages security events from IDS sensors (ISO 21434).

3.3 Service-Oriented Communication (SOME/IP)

The Adaptive Platform uses SOME/IP (Scalable service-Oriented MiddlewarE over IP) as its primary communication protocol over Ethernet. SOME/IP provides:

- **Service Discovery (SOME/IP-SD):** Dynamic runtime discovery of services. Adaptive Applications advertise their services and subscribe to services they need without static configuration. This enables truly dynamic software architectures.
- **Remote Procedure Calls (RPC):** Methods — callable functions exposed as services. Supports both fire-and-forget and request-response patterns.
- **Events:** Services can publish events that subscribers receive. Used for periodic data (e.g., object lists, ego motion) and change-driven data (e.g., mode changes, fault notifications).

- **Fields:** Combination of event and getter/setter methods. Represents a persistent value that can be read, written, and subscribed to.
- **Serialisation:** SOME/IP defines a binary serialisation format for all data types. The Adaptive Platform tooling generates serialisation code from ARXML service interface definitions.

3.4 Adaptive Application Structure

An Adaptive Application consists of one or more Processes. Each Process contains one or more Execution Contexts (threads). The Execution Management (EM) functional cluster controls the lifecycle of each Process based on the Machine State and Function Group States configured in the deployment.

- **Machine State:** Top-level state of the AUTOSAR Adaptive machine (e.g., Startup, Running, Shutdown, Update).
- **Function Group State:** State of a logical group of functions within the machine. Autonomous driving might define an AD Function Group with states: Off, Standby, Active, Degraded.
- **Supervised Entities:** PHM monitors application health through alive supervision (watchdog-like periodic reporting), deadline supervision (task completion within time bounds), and logical supervision (correct sequencing of checkpoints).

3.5 AP and Autonomous Driving

The Adaptive Platform is the primary AUTOSAR standard for autonomous driving ECUs (Central Computing Units/HPCs). A typical Level 2+ or Level 3 automated driving system uses:

- **Perception stack:** Camera, radar, LiDAR processing running as Adaptive Applications on the HPC. Object lists and free-space grids are published as SOME/IP events consumed by the fusion layer.
- **Sensor fusion:** Fuses object lists from multiple perception Adaptive Applications. Publishes fused object list as a SOME/IP service.
- **Prediction:** Subscribes to fused object list, predicts trajectories. Publishes as SOME/IP service.
- **Planning:** Subscribes to prediction outputs, ego motion, HD map. Computes trajectory and publishes as a SOME/IP service consumed by the motion controller.
- **Gateway to Classic Platform:** An Adaptive Application acts as a gateway, subscribing to planning outputs and translating them to Classic Platform signals for the brake and throttle controllers.

4. SOME/IP Communication Protocol

SOME/IP (Scalable service-Oriented MiddlewarE over IP) is the primary application-layer protocol for service-oriented communication in AUTOSAR Adaptive Platform and increasingly in AUTOSAR Classic Platform ECUs connected via Automotive Ethernet.

4.1 SOME/IP Message Structure

Every SOME/IP message has a fixed 16-byte header:

Field	Size	Description
Service ID	16 bits	Identifies the service (e.g., 0x0ABC = Object List Service)
Method ID	16 bits	Identifies the method/event/field within the service
Length	32 bits	Total length of the message in bytes (including header from byte 8)
Client ID	16 bits	Unique identifier of the calling client (used for request-response matching)
Session ID	16 bits	Sequence number for tracking request-response pairs
Protocol Ver	8 bits	SOME/IP protocol version (currently 0x01)
Interface Ver	8 bits	Service interface version (major version)
Message Type	8 bits	REQUEST, REQUEST_NO_RETURN, NOTIFICATION, RESPONSE, ERROR
Return Code	8 bits	E_OK, E_NOT_OK, E_UNKNOWN_SERVICE, E_UNKNOWN_METHOD, etc.

4.2 SOME/IP Service Discovery (SD)

SOME/IP-SD operates over UDP multicast (usually 224.224.224.245:30490). The SD protocol provides:

- **OfferService:** A service provider broadcasts that it is offering a service. Contains service ID, instance ID, major/minor version, and transport endpoint (IP:port).
- **FindService:** A client broadcasts that it is looking for a specific service. Service providers respond with OfferService if they match.
- **SubscribeEventgroup / SubscribeEventgroupAck:** Client subscribes to an event group (collection of related events/fields). Server acknowledges and starts sending events to the client's endpoint.
- **StopOfferService / StopSubscribeEventgroup:** Graceful teardown of services and subscriptions.

4.3 SOME/IP vs. Classic COM Signals

Classic Platform uses PDU-based signal communication with static configuration — every signal is pre-configured at compile time. SOME/IP-SD enables dynamic runtime discovery and subscription, which is essential for the flexible software architectures of autonomous driving HPCs where software modules may be loaded, updated, or restarted dynamically.

5. AUTOSAR and ISO 26262 Integration

AUTOSAR and ISO 26262 are complementary standards that work together in ADAS system development. AUTOSAR provides the software architecture framework; ISO 26262 provides the safety requirements that the AUTOSAR-based system must satisfy.

5.1 Key Integration Points

- **E2E Library:** AUTOSAR's E2E Library (Classic Platform) provides the CRC and sequence counter protection required by ISO 26262 for safety-critical signal transmission. E2E profiles (E2E Profile 1–7) provide varying levels of protection for different use cases.
- **Memory Protection:** AUTOSAR OS memory protection (MPU configuration) provides the freedom-from-interference between software of different ASIL levels required by ISO 26262 Part 6.
- **Watchdog Management (WdgM):** AUTOSAR Watchdog Manager implements the supervised entity concept (alive supervision, deadline supervision, logical supervision) that satisfies ISO 26262's requirement for periodic watchdog-based monitoring of safety-relevant software.
- **DEM integration:** The Diagnostic Event Manager provides the infrastructure for ISO 26262's requirement to detect, report, and respond to faults. ADAS safety monitors report their fault status through DEM events, which trigger FIM inhibition of dependent functions.
- **ASIL-partitioned OS:** AUTOSAR OS can be configured with memory and timing partitions such that ASIL D tasks are isolated from ASIL B or QM tasks, satisfying ISO 26262 freedom-from-interference.

5.2 AUTOSAR Adaptive Platform Safety

The Adaptive Platform's safety support is less mature than the Classic Platform but is actively evolving. Key safety mechanisms in the AP:

- **Platform Health Management (PHM):** Provides alive, deadline, and logical supervision of Adaptive Applications — equivalent to WdgM in the Classic Platform.
- **Error Handling:** The AP defines standardised error types and error handling patterns. `ara::core::Result` and `ara::core::ErrorCode` provide safe error propagation without exceptions.
- **ASIL decomposition in AP:** AP systems typically use ASIL decomposition — an ASIL D function is split between an ASIL B AP component and an ASIL B Classic Platform component with independence demonstrated at the system level.

6. Key AUTOSAR Terminology Reference

Term	Definition
SW-C	Software Component. Application unit that communicates through standardised ports.
RTE	Runtime Environment. Generated middleware layer connecting SW-Cs and BSW.
BSW	Basic Software. Standardised low-level software modules (OS, COM, DEM, etc.).
MCAL	Microcontroller Abstraction Layer. Hardware-specific drivers for peripherals.
ARXML	AUTOSAR XML. Description format for all AUTOSAR model elements and configurations.
PDU	Protocol Data Unit. Unit of data exchanged between communication modules.
COM	Communication module. Manages signal/PDU transmission and reception.
PduR	PDU Router. Routes PDUs between communication modules.
CanIf	CAN Interface. Abstraction layer between CAN driver and upper communication stack.
DEM	Diagnostic Event Manager. Manages fault events and DTC storage.
DCM	Diagnostic Communication Manager. Handles UDS diagnostic sessions.
FIM	Function Inhibition Manager. Enables/disables functions based on fault status.
NVM	NVRAM Manager. Manages persistent data in non-volatile memory.
WdgM	Watchdog Manager. Implements supervised entity monitoring (alive, deadline, logical).
E2E	End-to-End protection library. Adds CRC/sequence counter to safety-critical PDUs.
SOME/IP	Scalable service-Oriented MiddlewarE over IP. Service-oriented protocol over Ethernet.
SOME/IP-SD	SOME/IP Service Discovery. Dynamic discovery and subscription of SOME/IP services.
ARA	AUTOSAR Runtime for Adaptive Applications. Standardised API for Adaptive Platform.
ara::com	Adaptive Platform communication API. Service-oriented communication for Adaptive Apps.
PHM	Platform Health Management. Supervises Adaptive Application health (AP equivalent of WdgM).
EM	Execution Management. Controls process lifecycle in the Adaptive Platform.
UCM	Update & Configuration Management. OTA software update management.
AA	Adaptive Application. Application software running on the Adaptive Platform.
Machine	In AP: a logical unit of execution (ECU or VM) running the Adaptive Platform.
Functional Cluster	Group of related platform services in the Adaptive Platform.
gPTP	Generalized Precision Time Protocol (IEEE 802.1AS). Global time synchronisation over Ethernet.
DDS	Data Distribution Service. Alternative to SOME/IP for publish-subscribe communication (added in R22-11 CP, ...).
ASAM OpenX	Open standards for test scenario (OpenSCENARIO), road network (OpenDRIVE), and road surface (OpenCRG).

7. AUTOSAR Architecture Patterns in ADAS Systems

7.1 Typical BMW/OEM ADAS ECU Stack (Classic Platform)

A typical BMW ADAS ECU (e.g., the central safety domain controller hosting ACC, AEB, LKA) uses an AUTOSAR Classic Platform stack with the following key architectural characteristics:

- **Top-level SW-Cs:** AccController_SwC, AebController_SwC, LkaController_SwC, and SafetyMonitor_SwC communicate through RTE sender-receiver ports. The SafetyMonitor_SwC subscribes to all safety-relevant signals and reports to DEM.
- **Codec framework:** In multi-version vehicle programmes, a codec layer SW-C handles protocol differences between vehicle generations, translating between different message formats while the core ADAS logic remains version-independent.
- **Signal degradation:** A dedicated SignalDegradation_SwC monitors signal health qualifiers, sequence counters, and E2E status for all ASIL-relevant inputs. Upon degradation, it sets DEM events which FIM uses to inhibit affected ADAS functions gracefully.
- **Version handling:** Template-based C++17 mechanisms (in post-Classic-only implementations) or #ifdef-based Bazel build configurations handle multi-platform support within a single codebase.

7.2 Transition from Classic to Adaptive Platform

The automotive industry is progressively migrating compute-intensive ADAS functions to the Adaptive Platform on HPCs while retaining safety-critical actuation control on Classic Platform ECUs. This creates a heterogeneous architecture:

- **HPC (Adaptive Platform):** Sensor fusion, object detection (camera/LiDAR neural networks), prediction, path planning. These run as Adaptive Applications communicating via SOME/IP services.
- **Safety ECU (Classic Platform):** Brake controller, steering controller, throttle controller. Receives actuation commands from HPC via Ethernet gateway. Applies E2E protection and independent safety monitoring.
- **Gateway ECU:** Translates between HPC SOME/IP messages and Classic Platform CAN signals. Performs protocol adaptation while maintaining E2E protection across the boundary.
- **Safety decomposition:** The overall system safety case is built by combining the AP's ASIL B monitoring with the Classic Platform's independent ASIL B actuation safety, achieving an effective ASIL D system-level requirement through decomposition.